

深度学习作业 2

221900180 田永铭 tianym2022@smail.nju.edu.cn

2024 年 11 月 17 日

一、循环神经网络 (RNN)

Problem 1.1 vanilla 的 RNN 为什么会出现梯度消失或梯度爆炸的情况？

Solution: vanilla 的 RNN 在训练过程中需要通过时间展开计算长序列的梯度。RNN 的训练通过时间展开后，将每个时间步的计算视为一个层，并利用反向传播算法更新权重。对于长序列的梯度计算，误差信号需要从较晚的时间步向较早的时间步逐步传播，梯度的计算涉及多次链式相乘。我们将梯度具体表达式写出来：

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}, \quad \frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh}h_{t-1} + W_{xh}x_t) W_{hh}$$
$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_T}{\partial W} \cdots \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}} \right) \frac{\partial h_1}{\partial W}.$$

可以看到，梯度更新时关键的一项是连乘项，而连乘的因子具有这样的性质：

1. 如果激活函数 σ 是类似 $\tanh$ 这样的满足 $\sigma' < 1$ 的，则累乘后一般会使得整个式子越来越小，接近为 0，梯度几乎不动，造成了**梯度消失**。而如果权重矩阵 $\mathbf{W}$ 的最大奇异值远远大于 1，即 $\|\mathbf{W}\| \gg 1$ 的时候，这个乘积项因子可能更会大于 1，累成后越来越大，一定可能会造成**梯度爆炸**。
2. 如果我们去除非线性激活函数，此时的情况就比较绝对了：累乘后会出现一个 $\mathbf{W}_{hh}^{T-1}$ 的指数次方，若权重矩阵 $\mathbf{W}$ 的最大奇异值大于 1，即 $\|\mathbf{W}\| > 1$ ，则累成后**梯度爆炸**。若权重矩阵 $\mathbf{W}$ 的最大奇异值小于 1，即 $\|\mathbf{W}\| < 1$ ，则**梯度消失**。

Problem 1.2 vanilla RNN 出现梯度爆炸该如何解决？

Solution: 解决梯度爆炸有多种方法，除了课上提及的梯度剪裁，还有其它一些方法，我将给出方法以及自己对这些方法的理解，分别如下：

1. 梯度裁剪（我理解为直接暴力地将 W 变小来使得累乘项变小）：在反向传播时，当梯度范数超过某个阈值时对梯度进行缩放：

$$g \leftarrow g \cdot \frac{\text{threshold}}{\|g\|_2}, \quad \text{if } \|g\|_2 > \text{threshold}.$$

2. 正则化（我理解为通过优化间接地将 W 变小来使得累乘项变小）：在损失函数中添加权重衰减项：

$$L = L_{\text{data}} + \lambda \|W\|_2^2,$$

其中 λ 为正则化系数。也可以采用 Dropout，在隐藏层中随机丢弃神经元，减少权重的增长。

3. 合适的权重初始化（我理解为合适的权重初始化保持了激活值和梯度的方差稳定，不会集中增大从而爆炸）：比如 Xavier 初始化：保证输入和输出的方差一致：

$$W \sim \mathcal{U}\left(-\sqrt{\frac{1}{n}}, \sqrt{\frac{1}{n}}\right),$$

其中 n 是输入节点数。以及 He 初始化，适合 ReLU 激活函数：

$$W \sim \mathcal{N}\left(0, \frac{2}{n}\right).$$

4. 优化器选择（我理解为降低学习率能缓解更新的程度，但是效果应该不会很显著）：使用高级优化器（如 Adam 或 RMSprop）代替 SGD，这些优化器可以动态调整学习率，对梯度的变化更为鲁棒。

5. 梯度归一化（我理解为和梯度剪裁类似）：对梯度进行归一化处理：

$$g \leftarrow \frac{g}{\|g\|_2 + \epsilon}.$$

6. 改进模型结构（这个后面题目也会说到）：使用 LSTM 或 GRU 替代 Vanilla RNN，通过引入门机制控制信息流动，避免梯度爆炸。

7. 截断反向传播（我理解为减少了累乘的长度）：在处理长序列时，可以将序列分段进行梯度计算，而不是反向传播整个序列。这样可以减小梯度爆炸的可能性

Problem 1.3 LSTM 是如何缓解梯度消失的，它能彻底解决梯度消失吗？

Solution: LSTM 的核心思想是通过其门控机制（输入门、遗忘门和输出门）和记忆单元（cell state），在信息的存储和传递过程中实现了更高效的梯度流动，缓解了梯度消失问题。具体图示和做法如下：

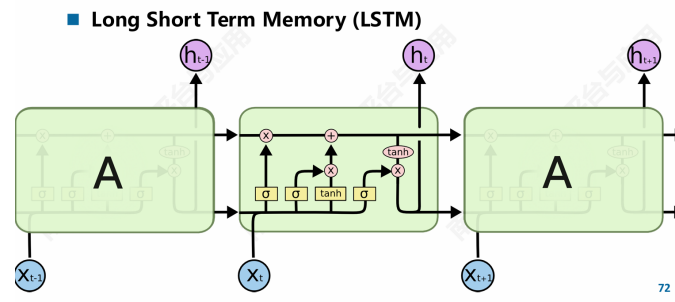


图 1: LSTM 单元结构

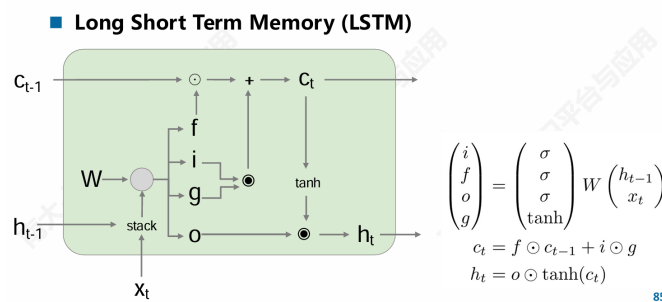


图 2: LSTM 计算公式

1. **输入门 (Input Gate):** 输入门决定当前输入 x_t 如何影响记忆单元。它的计算公式为：

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

其中， σ 是 sigmoid 激活函数， W_i 是输入门的权重矩阵， h_{t-1} 是上一个时间步的隐藏状态， x_t 是当前时刻的输入， b_i 是偏置项。

2. **遗忘门 (Forget Gate):** 遗忘门决定当前时刻的记忆单元应该丢弃多少信息。它的计算公式为：

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

其中， f_t 是遗忘门的输出，表示遗忘的程度， W_f 是遗忘门的权重矩阵， b_f 是偏置项。激活函数 σ 用的是 sigmoid，根据老师所讲，这里的道理是 sigmoid 的值在 0 和 1 之间，所以乘后一定会减小本来的值，起到“遗忘”的作用。

3. **更新记忆单元 (Cell State):** 记忆单元通过输入门和遗忘门控制信息的存储与丢弃。这个为存储长时记忆提供了手段。记忆单元的更新过程如下:

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

其中, $\tilde{C}_t$ 是候选记忆单元, C_t 是更新后的记忆单元, W_C 是记忆单元的权重矩阵, b_C 是偏置项。

4. **输出门 (Output Gate):** 输出门决定记忆单元的哪些部分将影响当前的输出 h_t 。它的计算公式为:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \cdot \tanh(C_t)$$

其中, o_t 是输出门的输出, W_o 是输出门的权重矩阵, b_o 是偏置项, h_t 是当前时间步的隐藏状态输出。

LSTM 不能彻底解决梯度消失。主要由于两点:

1. LSTM 的门控机制虽然可以调节梯度, 但在面对极长序列时, 依然可能会遇到梯度消失的情况。尤其在训练时, 梯度可能经过多次乘法, 导致它仍然会衰减到几乎为零。
2. LSTM 的结构仍然包含非线性激活函数 (如 $\tanh$ 或 sigmoid), 这些函数在极端值区域可能会导致梯度过小, 从而加剧梯度消失。

更根本的, 我们从梯度消失的根本原因出发, 参考问题1.1, LSTM 只能缓解梯度消失, 并不能解决它。

二、注意力（Attention）

Problem 2.1 完成下面 self-attention 的代码填空，并结合 attention 计算的数学公式解释你的做法。

```
1 import torch
2 import torch.nn.functional as F
3 # 假设输入是一个5个词，4维的词向量矩阵
4 X = torch.tensor([[1.0, 0.0, 1.0, 0.0],
5                   [1.0, 2.0, 1.0, 0.0],
6                   [0.0, 1.0, 0.0, 1.0],
7                   [0.0, 1.0, 0.0, 0.0],
8                   [1.0, 0.0, 1.0, 0.0]])
9 # 初始化查询、键和值的权重矩阵
10 W_Q = torch.randn(4, 4)
11 W_K = torch.randn(4, 4)
12 W_V = torch.randn(4, 4)
13
14 # todo 计算查询、键、值
15
16 # todo 计算注意力得分
17
18 # todo 加权和计算最终output
19
20 # print(output)
```

Solution: 代码填空如下：

```
1 import torch
2 import torch.nn.functional as F
3
4 # 假设输入是一个5个词，4维的词向量矩阵
5 X = torch.tensor([[1.0, 0.0, 1.0, 0.0],
6                   [1.0, 2.0, 1.0, 0.0],
7                   [0.0, 1.0, 0.0, 1.0],
8                   [0.0, 1.0, 0.0, 0.0],
9                   [1.0, 0.0, 1.0, 0.0]])
10
11 X = X.T # 注意我把X转置了！！！！！！
12
```

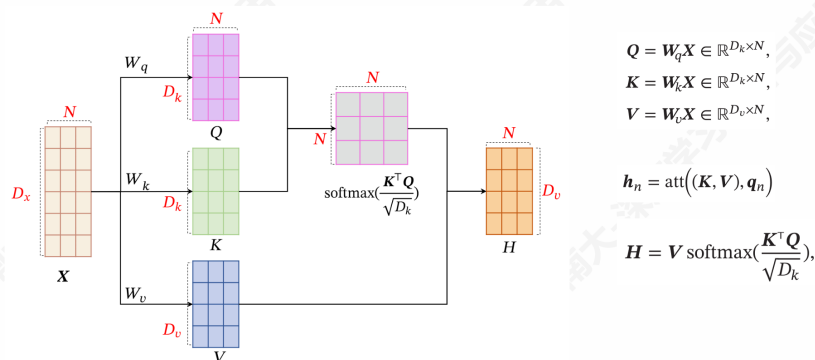
```

13 # 初始化查询、键和值的权重矩阵
14 W_Q = torch.randn(4, 4)
15 W_K = torch.randn(4, 4)
16 W_V = torch.randn(4, 4)
17
18 # 计算查询、键、值
19 Q = torch.matmul(W_Q, X) # 查询矩阵
20 K = torch.matmul(W_K, X) # 键矩阵
21 V = torch.matmul(W_V, X) # 值矩阵
22
23 # 计算注意力得分
24 attn_scores = torch.matmul(K.T, Q) # Q 和 K 的点积
25 attn_scores = attn_scores / torch.sqrt(torch.tensor(Q.size(0), dtype=torch.
    float32)) # 缩放
26
27 # 通过 softmax 进行归一化
28 attn_probs = F.softmax(attn_scores, dim=-1)
29
30 # 加权和计算最终 output
31 output = torch.matmul(V, attn_probs)
32
33 print(output)

```

以上的代码完全按照老师 PPT 提供的 attention 的计算公式实现，公式如下：

■ 自注意力的 Q-K-V



19

图 3: Formulas

具体步骤和对应关系如下：

0. 首先将 X 转置, 因为老师 PPT 图中 X 的应该是 $D_x \times N$ 的, 但是输入的 X 相反了, 所以要转置一下! 对应代码11行。

1. 查询、键、值的计算:

$$Q = W_Q X, \quad K = W_K X, \quad V = W_V X$$

其中, W_Q, W_K, W_V 分别是查询、键、值的权重矩阵。对应代码14到16行。

2. 计算注意力得分:

$$\text{attn_scores} = K^T Q$$

缩放因子:

$$\text{attn_scores} = \frac{K^T Q}{\sqrt{d_k}}, \quad \text{其中 } d_k \text{ 是查询向量的维度, 也就是 } Q \text{ 的行数}$$

对应代码24到25行。

3. Softmax: 对每个查询的注意力得分进行 Softmax 归一化, 得到注意力权重:

$$\text{attn_probs} = \text{Softmax}(\text{attn_scores})$$

对应代码28行。

4. 加权求和计算最终输出:

$$\text{output} = V \cdot \text{attn_probs}$$

对应代码33行。

运行结果产生 output 的如下 (这个结果由于随机初始化等等所以是随机的, 仅供参考):

```
tensor([[ 0.4478,  0.9495,  3.1809,  1.3268,  0.4478],
        [ 0.1962,  0.3856,  1.2222,  0.5758,  0.1962],
        [ 0.1040, -0.0889,  0.4146,  0.2687,  0.1040],
        [ 0.1201,  0.6565,  0.4415,  0.3969,  0.1201]])
```

Problem 2.2 对比并简要解释多头注意力的实现方式。

Solution: 单头注意力 (Scaled Dot-Product Attention) :

单头注意力机制计算参考图3。

在单头注意力中，模型通过一个单独的查询、键和值矩阵进行计算，输出是基于整个输入的加权和。

多头注意力 (Multi-Head Attention) :

与单头注意力不同，多头注意力通过将输入的查询、键和值矩阵分别线性变换为多个子空间，从而允许模型同时关注不同的表示子空间。

首先，将输入的 Q 、 K 和 V 通过线性变换映射到多个头的查询、键和值：

$$Q_i = QW_i^Q, \quad K_i = KW_i^K, \quad V_i = VW_i^V$$

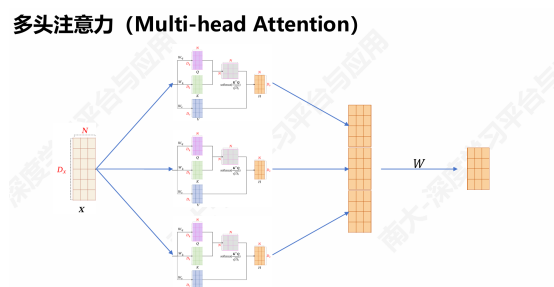
每个头 i 独立地计算注意力：

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i)$$

然后，将所有头的输出拼接起来，并通过一个线性变换得到最终的输出：

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h) W^O$$

其中， W^O 是输出的投影矩阵。图示如下：



对比：

单头注意力：仅通过一个查询、键和值计算注意力，并返回加权和。其表达能力受限于一个线性变换的结果。

多头注意力：通过多个头并行计算注意力，每个头独立地学习不同的表示子空间。最后将各个头的输出拼接，并进行线性变换，能够捕捉到更多的关系和模式。

Problem 2.3 Transformer 结构中 encoder 和 decoder 的区别有哪些?

Solution: 我认为区别主要在以下几个方面:

1. 功能:

- **Encoder:** 负责处理输入数据 (如句子), 将其转换为一个上下文相关的表示 (即隐藏状态), 这个表示被传递给 Decoder。
- **Decoder:** 生成输出数据 (如翻译的句子), 在生成过程中依赖于 Encoder 的输出和自身已经生成的部分。

2. 输入和输出

- **Encoder:** 输入是原始的序列数据, 输出是对输入序列的深度表示 (即隐层表示)。
- **Decoder:** 输入包括 Encoder 的输出 (编码后的表示) 和 Decoder 自身已经生成的部分。Decoder 的输出是一个概率分布, 用于生成下一个单词或符号。

3. 自注意力机制是否有遮蔽

- **Encoder:** 每个 Encoder 层内有一个自注意力层, 它通过关注输入序列的所有位置, 计算每个位置对其他位置的依赖关系。自注意力机制没有遮蔽。
- **Decoder:** Decoder 也有自注意力层, 但与 Encoder 不同, Decoder 的自注意力机制是有遮蔽的 (Masked), 即每个位置只能看到它自己以及之前的输出, 以防止模型在生成时“偷看”未来的信息。

4. 是否有 Encoder-Decoder 注意力机制

- **Encoder:** 没有这个机制。Encoder 只关注输入序列。
- **Decoder:** Decoder 的每一层都有一个 Encoder-Decoder 注意力层, 它允许 Decoder 在生成过程中, 利用 Encoder 输出的上下文信息来调整生成的内容。

5. 层数和复杂度

- **Encoder 和 Decoder** 的结构基本相同, 通常由多层 (如 6 层) 的 Encoder 和 Decoder 堆叠而成, 每一层都包含自注意力机制、前馈神经网络以及在 Decoder 中的 Encoder-Decoder 注意力机制层。Decoder 通常会比 Encoder 多一个额外的注意力机制层。

三、目标检测（R-CNN）

Problem 3.1 请简要描述 R-CNN 模型的工作流程。具体来说，R-CNN 是如何进行目标检测的？简述 R-CNN 模型的主要步骤。

Solution: 这个几个问题实际上都是问的一个问题，即 R-CNN 的工作步骤流程，流程如下：

1. **区域提议生成:** R-CNN 首先使用一个选择性搜索（Selective Search）算法从输入图像中用**区域生成网络**生成多个候选区域（Regions of Interest (ROI)）。这些区域可能包含目标物体或背景。选择性搜索通过多种策略（如颜色、纹理、大小和形状等）将图像分割成若干小区域，然后将这些区域逐渐合并，生成不同大小的候选区域。
2. **特征提取:** 每个图像区域使用**卷积网络处理 ImageNet 预训练的 CNN**。通常，使用的是像 AlexNet 这样的深度卷积神经网络，CNN 会为每个候选区域提取一个固定长度的特征向量（通常是一个高维向量）。这些特征向量能很好地表示区域内部的视觉信息。
3. **分类与回归:** 对于每个候选区域的特征向量，R-CNN 使用支持向量机（SVM）进行分类，判断该区域是否包含某个特定目标类别（例如人、车、猫等）。这个过程对于每个类别都需要**训练一个独立的 SVM**。同时，R-CNN 还会进行边界框回归（Bounding Box Regression），即预测该候选区域的准确边界框位置，通过微调候选区域的位置和大小，使其更精确地覆盖物体。即对每个 **RoI** 预测调整值。
4. **最终检测结果输出:** 对于所有候选区域，R-CNN 会得到类别预测和边界框回归结果。然后，使用非极大值抑制（Non-Maximum Suppression, NMS）来去除重复的检测框，保留最准确的目标检测结果。

Problem 3.2 与传统的 R-CNN 相比，Fast R-CNN 的最大改进是什么？请简要说明 Fast R-CNN 如何避免了 R-CNN 中存在的冗余计算问题。

Solution: 最大改进:

传统 R-CNN 使用 SVM 对每个 ROI 分类的时候，有些区域会重复计算多次（被多个 ROI 覆盖的），从而产生“效率很低，非常慢”的问题。而 Fast R-CNN 的最大改进就是利用 idea: “使用卷积网络处理整张图像，在特征上进行区域提取”，大大减少了重复计算，从而大大提高了处理速度。数值上，Fast R-CNN 训练非常深的 VGG16 网络比 R-CNN 快 9 倍，在测试时快 213 倍。

具体如何避免了冗余计算:

1. **使用卷积网络处理整张图像:** 传统 R-CNN 一张图像内候选框之间大量重叠，提取特征操作冗余。而 Fast R-CNN 整个图像只需要通过一次卷积神经网络（CNN）。Fast

R-CNN 直接对整个图像计算卷积特征，然后通过区域提议（Region of Interest, RoI）将卷积特征映射到每个候选区域，大大减少了重复计算。

2. 用 **ROI pooling** 层取代最后一层 **max pooling** 层：Fast R-CNN 将分类和边界框回归任务统一为一个网络，在共享特征基础上完成。避免冗余计算的核心机制是 Fast R-CNN 引入了 RoI 池化层（Region of Interest Pooling Layer），其关键在于：首先对整个输入图像执行一次 CNN 前向传播，生成共享的卷积特征图。对每个候选区域，通过 RoI 池化层将该区域在特征图上的对应部分提取出来，转化为固定大小的特征。这些固定大小的特征被用于后续的分类和边界框回归任务。

除了避免冗余计算之外，Fast R-CNN 效率的提高还一部分在于实现了端到端的训练：Fast R-CNN 网络末尾采用并行的不同的全连接层，可同时输出分类结果和窗口回归结果，实现了 end-to-end 的多任务训练，也不需要额外的特征存储空间。